

Create 2 EC2 instances on 2 different regions and install nginx using terraform script.

```
vicky@SriJay:~/Terraform$ vi main.tf
vicky@SriJay:~/Terraform$ cat main.tf
#####
# Providers
#####

provider "aws" {
  alias = "south"
  region = "ap-south-1"
}

provider "aws" {
  alias = "west"
  region = "us-west-2"
}

#####
# EC2 in ap-south-1
#####

resource "aws_instance" "South_instance" {
  ami           = "ami-019715e0d74f695be" # Amazon Linux 2 (ap-south-1)
  instance_type = "t3.micro"
  user_data = <<-EOF
    #!/bin/bash
    sudo apt update -y
    sudo apt install nginx -y
    sudo systemctl start nginx
    sudo systemctl enable nginx
  EOF

  tags = {
```

```

vicky@SriJay: ~/Terraform
}
}

#####
# EC2 in us-west-2
#####

resource "aws_instance" "west_instance" {
  provider      = aws.west
  ami           = "ami-0786adace1541ca80" # Amazon Linux 2 (us-west-2)
  instance_type = "t3.micro"
  user_data = <<-EOF
    #!/bin/bash
    sudo apt update -y
    sudo apt install nginx -y
    sudo systemctl start nginx
    sudo systemctl enable nginx
  EOF

  tags = {
    Name = "west-Nginx-ec2"
  }
}

vicky@SriJay:~/Terraform$ terraform init
Command 'terraform' not found, did you mean:
  command 'terraform' from snap terraform (1.14.3)
See 'snap info <snapname>' for additional versions.
vicky@SriJay:~/Terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.31.0

```

```
vicky@SriJay:~/Terraform$ terraform init
Command 'terraform' not found, did you mean:
  command 'terraform' from snap terraform (1.14.3)
See 'snap info <snapname>' for additional versions.
vicky@SriJay:~/Terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.31.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
vicky@SriJay:~/Terraform$ terraform plan
aws_instance.west_instance: Refreshing state... [id=i-0e621cd45b20aeb6b]

Terraform used the selected providers to generate the following execution plan.
following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.South_instance will be created
+ resource "aws_instance" "South_instance" {
```

```
vicky@SriJay:~/Terraform$ terraform plan
aws_instance.west_instance: Refreshing state... [id=i-0e621cd45b20aeb6b]

Terraform used the selected providers to generate the following execution plan. Resources
following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.South_instance will be created
+ resource "aws_instance" "South_instance" {
  + ami                        = "ami-019715e0d74f695be"
  + arn                       = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone          = (known after apply)
  + disable_api_stop           = (known after apply)
  + disable_api_termination    = (known after apply)
  + ebs_optimized              = (known after apply)
  + enable_primary_ipv6        = (known after apply)
  + force_destroy              = false
  + get_password_data          = false
  + host_id                    = (known after apply)
  + host_resource_group_arn    = (known after apply)
  + iam_instance_profile       = (known after apply)
  + id                         = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle         = (known after apply)
  + instance_state             = (known after apply)
  + instance_type              = "t3.micro"
```

```
vicky@SriJay:~/Terraform$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.South_instance will be created
+ resource "aws_instance" "South_instance" {
+   ami                     = "ami-019715e0d74f695be"
+   arn                     = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone        = (known after apply)
+   disable_api_stop         = (known after apply)
+   disable_api_termination  = (known after apply)
+   ebs_optimized            = (known after apply)
+   enable_primary_ipv6       = (known after apply)
+   force_destroy            = false
+   get_password_data         = false
+   host_id                  = (known after apply)
+   host_resource_group_arn   = (known after apply)
+   iam_instance_profile      = (known after apply)
+   id                       = (known after apply)
+   instance_initiated_shutdown_behavior = (known after apply)
+   instance_lifecycle        = (known after apply)
+   instance_type             = (known after apply)
+   key_name                  = (known after apply)
+   monitoring                 = (known after apply)
+   network_interface         = (known after apply)
+   placement_group           = (known after apply)
+   placement_group_name      = (known after apply)
+   placement_group_strategy  = (known after apply)
+   primary_monitoring        = (known after apply)
+   profile                    = (known after apply)
+   subnet_id                 = (known after apply)
+   tags                      = {}
+   user_data                 = (known after apply)
+   vpc_security_group_ids    = (known after apply)
}
```

West region Ec2 created

Instances (1) [Info](#) Last updated less than a minute ago [Refresh](#) [Connect](#) [Instance state](#) [Actions](#)

[All state](#)

[Instance state = running](#) [Clear filters](#)

☐	Name ✎	Instance ID	Instance state	Instance type
☐	west-Nginx-ec2	i-00072be934789a7f7	Running	t3.micro

Instance summary for i-00072be934789a7f7 (west-Nginx-ec2) [Info](#)

[Refresh](#) [Connect](#) [Instance state](#) [Actions](#)

Updated less than a minute ago

Instance ID
i-00072be934789a7f7

IPv6 address
-

Hostname type
IP name: ip-172-31-19-225.us-west-2.compute.internal

Public IPv4 address
35.85.50.154 | [open address](#)

Instance state
Running

Private IP DNS name (IPv4 only)
ip-172-31-19-225.us-west-2.compute.internal

Private IPv4 addresses
172.31.19.225

Public DNS
ec2-35-85-50-154.us-west-2.compute.amazonaws.com

Access web browser on west EC2

 Not secure 35.85.50.154

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Created south region EC2

Instance summary for i-031410ceedbf2a85c (South-Nginx-ec2) [Info](#)



Connect



Instance

Updated 3 minutes ago

Instance ID

 i-031410ceedbf2a85c

IPv6 address

—

Hostname type

IP name: ip-172-31-45-10.ap-south-1.compute.internal

Public IPv4 address

 3.108.227.139 | [open address](#)

Instance state

 Running

Private IP DNS name (IPv4 only)

 ip-172-31-45-10.ap-south-1.compute.internal

Private IPv4 addresses

 172.31.45.10

Public DNS

 ec2-3-108-227-139.ap-south-1.compute.amazonaws.com | [copy](#)

Access web browser on south EC2

 Not secure 3.108.227.139

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

